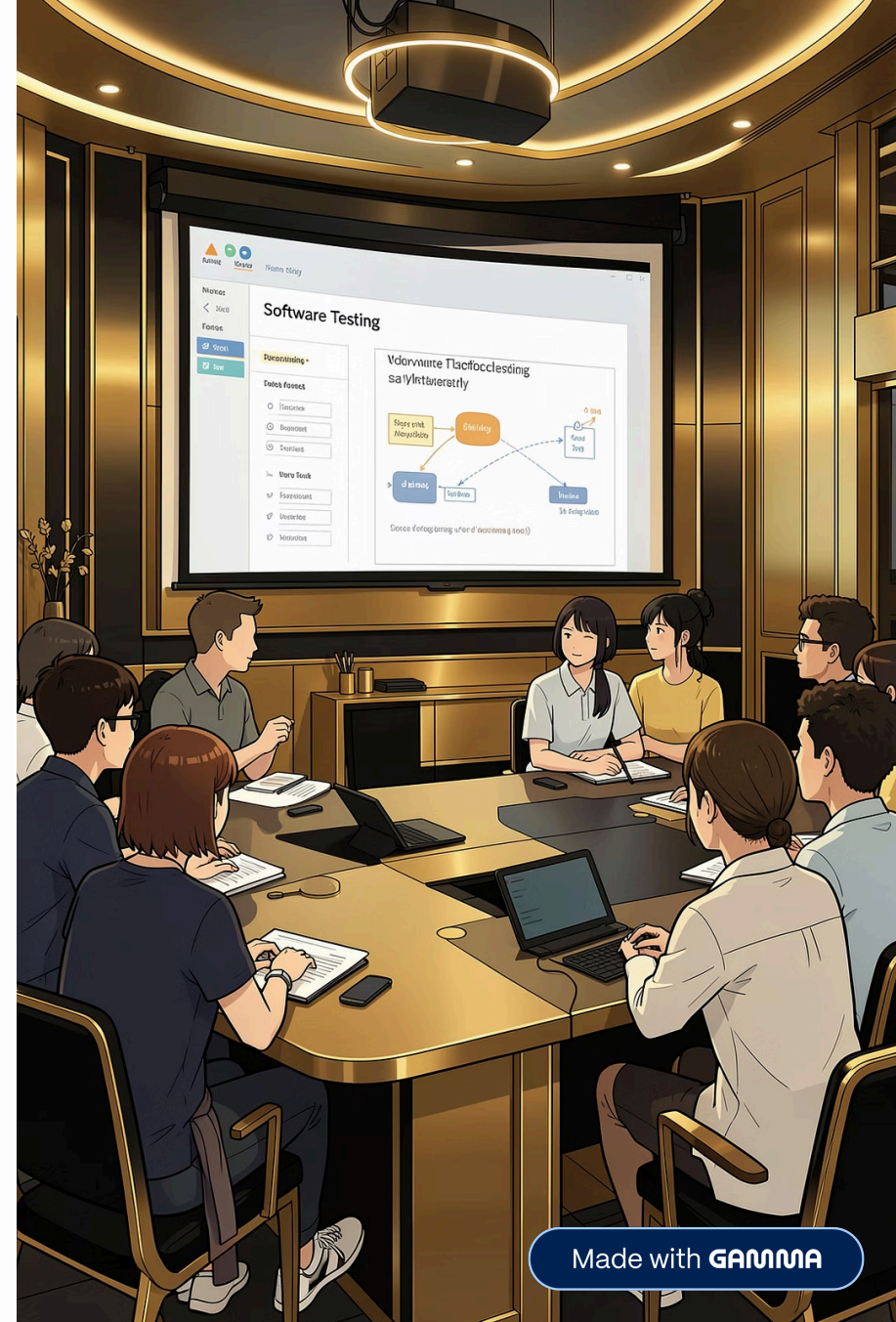


Unidade Curricular 3 — Testes e Manutenção de Sistemas de Informação

Introdução à certificação ISTQB, papel dos testes em qualidade de software e motivação para a disciplina. Público: estudantes de Sistemas de Informação e Engenharia de Software.



WELLINGTON ANTHONY SPERANSA

- GRADUADO EM GESTÃO DA TECNOLOGIA DA INFORMAÇÃO
- MANUTENÇÃO E SUPORTE TÉCNICO EM INFORMÁTICA
- JOVEM PROGRAMADOR 2025
- ADMINISTRAÇÃO DE REDES LINUX
- CAMPEÃO DA PRÊMIO ATENA

Indicadores

1

Utiliza diferentes metodologias de testes

De acordo com a funcionalidade do sistema.

2

Tabula o resultado

De acordo com as informações obtidas no teste realizado.

3

Realiza ajustes e ações corretivas

Conforme a análise do resultado coletado.

Conhecimentos

1

Plano de teste

metodologias, especificações do teste, defeito, falha, registros de teste.

2

Controle de versão

conceito; segurança da informação; instalação e configuração.

3

Ramificação do projeto

controle de histórico, marcações, resgate e versões estáveis.

4

Política de recuperação de dados

conceito; backup e restore.

5

Ciclo de vida do sistema

planejamento de atualização e correções.

Habilidades

1

Comunicar-se de maneira assertiva.

2

Identificar os aspectos do próprio trabalho que interferem na organização.

3

Efetuar cálculos das quatro operações matemáticas.

4

Resolver problemas lógicos e aritméticos.

5

Interpretar textos e manuais.

6

Elaborar relatórios e documentos técnicos.

7

Elaborar planos de teste.

Atitudes/Valores

1

Colaboração no desenvolvimento do trabalho em equipe.

2

Proatividade no desenvolvimento do trabalho.

3

Sigilo no tratamento de dados e informações.

4

Zelo na apresentação pessoal e postura profissional.

Me conte um pouco de você?

É um prazer tê-los conosco! Para começarmos, gostaríamos de conhecer um pouco mais sobre cada um de vocês. Compartilhem suas histórias e aspirações conosco.



Seu Nome

Como você gostaria de ser chamado(a)?



Sua Origem

De onde você vem e o que te traz aqui?



Experiência Tech

Qual sua vivência com programação e tecnologia?



Sua Motivação

O que te impulsiona a estudar testes de software?



Objetivos Futuros

Quais são seus objetivos profissionais?



O que é o ISTQB?

É a principal certificação internacional que padroniza o conhecimento e as práticas em testes de software.

Padronização

Principal certificação internacional que uniformiza termos, processos e práticas de testes.

Níveis

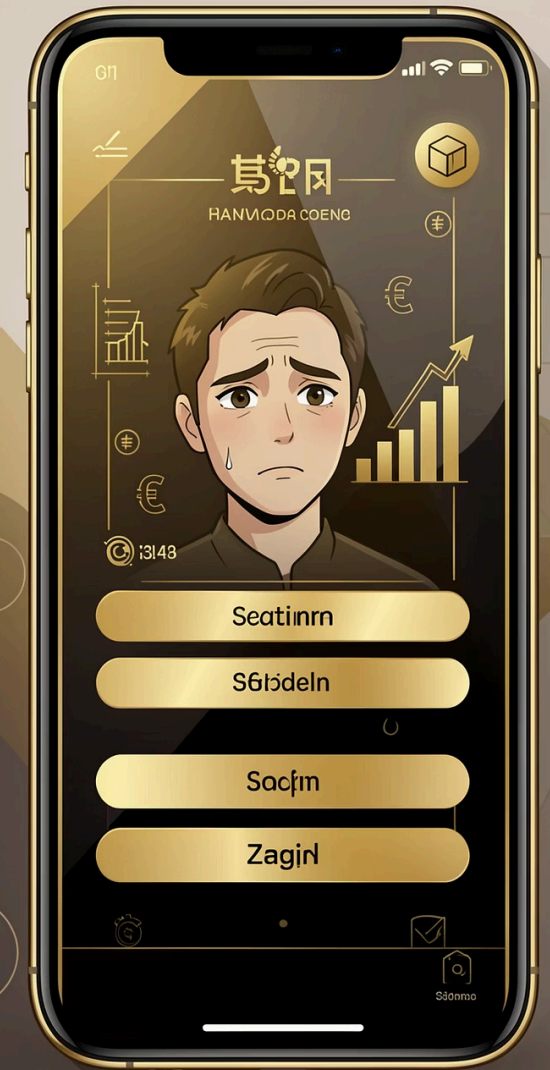
Foundation → Advanced → Expert: progressão modular para carreira em QA.

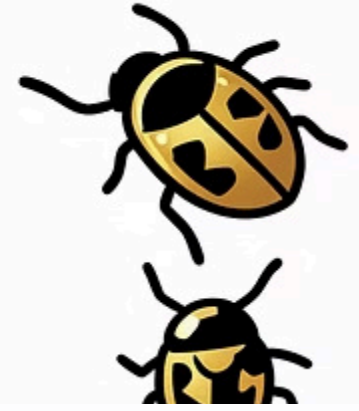
Benefícios

Melhora comunicação entre equipes, aumenta confiança de mercado e direciona melhores práticas.

Exemplo: Sistemas Bancários — confiança e risco

Os sistemas bancários exigem testes rigorosos: disponibilidade, segurança, integridade das transações e conformidade. **Confiariam em um app bancário sem testes automatizados e auditoria?** Quais os riscos reais (financeiros e reputacionais).

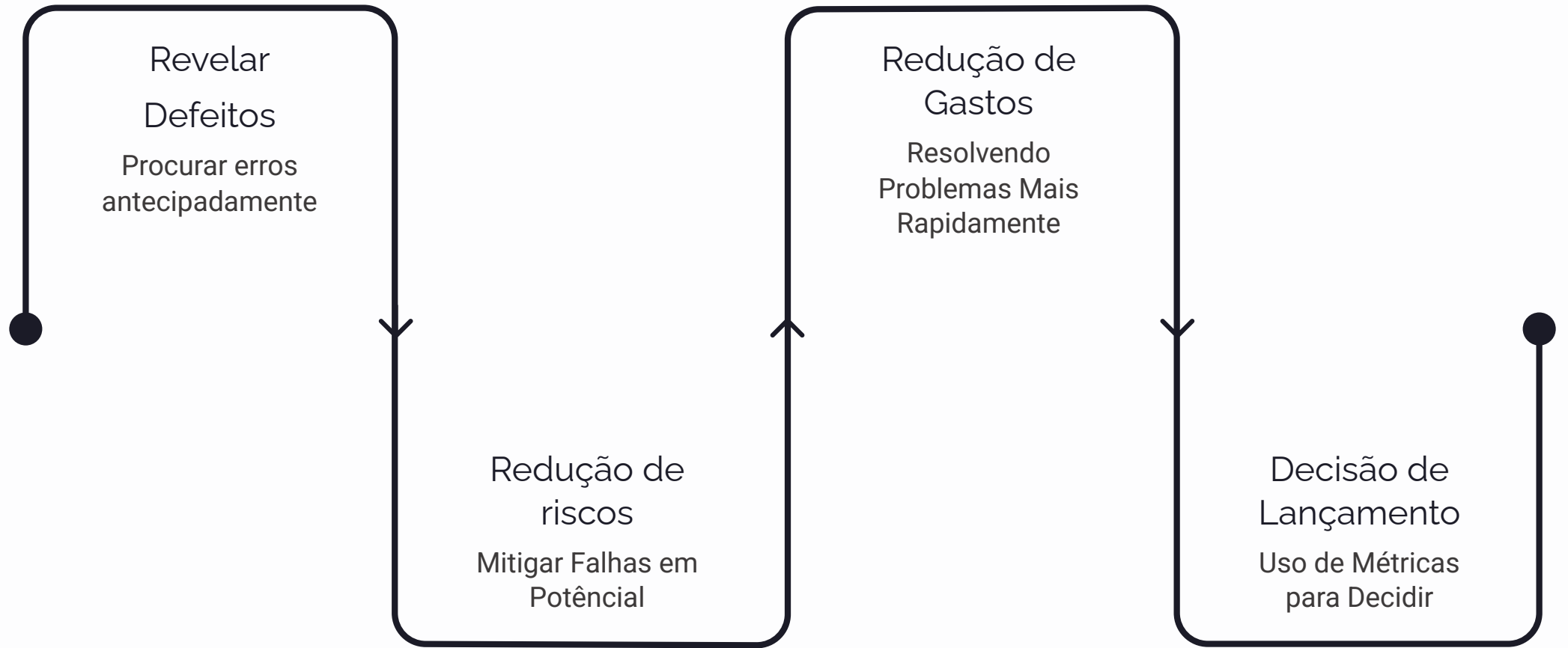




O que é Teste de Software?

Definição simples: testar é verificar se o sistema funciona como deveria. Script de apresentação: “Testar não é achar erro. Testar é garantir qualidade.”

❏ Citação ISTQB: testes revelam defeitos, reduzem riscos e apoiam decisões de liberação.



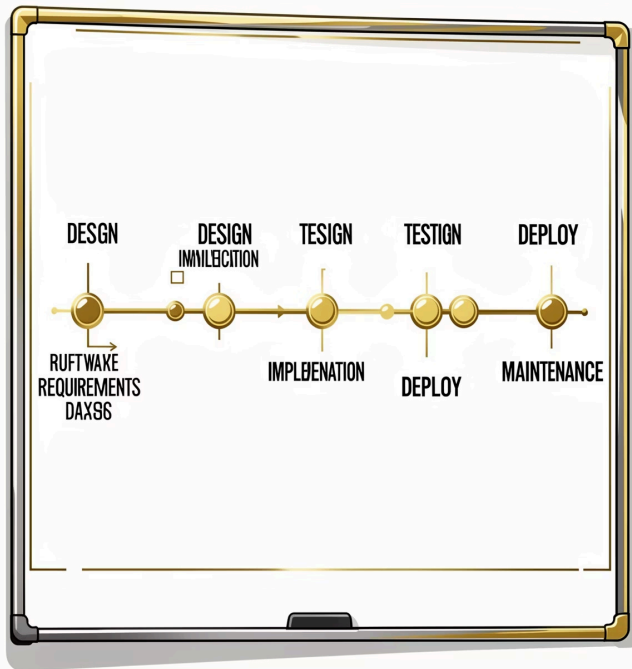
Aplicar testes traz: detecção de defeitos, mitigação de riscos, economia ao corrigir cedo e métricas para decidir liberação.

Porque aplicar testes de software?

“Testar é necessário para revelar defeitos presentes no software, reduzir riscos associados à entrega de produtos com falhas, aumentar a confiança na qualidade do sistema e apoiar a tomada de decisão quanto à liberação do produto. Além disso, testes realizados desde as fases iniciais do ciclo de vida promovem a economia de custos e a melhoria contínua.”— ISTQB® Foundation Level Syllabus

1. Garantia da Qualidade: Testes asseguram que o sistema atende aos requisitos funcionais e não funcionais definidos.
2. Redução de Riscos: Identifica defeitos antes que causem falhas graves em produção (perda financeira, reputação, segurança).
3. Correção Precoce: Quanto mais cedo um defeito é encontrado (ex: na fase de requisitos), menor o custo para corrigi-lo.
4. Confiabilidade do Produto: Sistemas testados geram mais confiança no usuário final, aumentando adoção e satisfação.
5. Base para Decisão de Liberação: Indicadores e métricas de teste (ex: cobertura, bugs críticos) apoiam decisões de ir ou não para produção.
6. Validação de Mudanças: Testes de regressão garantem que alterações no código não quebram funcionalidades já existentes.
7. Conformidade com Normas: Muitas indústrias (financeira, médica, automotiva) exigem testes documentados para compliance e auditorias.
8. Melhoria Contínua: Testes ajudam a refinar processos de desenvolvimento e promover a cultura de qualidade.

Quando testar no ciclo de vida?



- Durante a análise de Requisitos: Detecta ambiguidade, inconsistência ou ausência de requisitos.
- Durante o Design técnico: Evita falhas estruturais e decisões técnicas incorretas.
- Durante o desenvolvimento: Garante que cada módulo funcione isoladamente e conforme o esperado.
- Após a integração de módulos: Verifica se os componentes interagem corretamente entre si.
- Após o sistema estar completo: Valida o sistema como um todo, funcional e não funcionalmente.
- Antes da entrega ao cliente: Confirma que o sistema atende aos requisitos do usuário final.
- Após correções ou mudanças: Garante que novas alterações não quebraram funcionalidades existentes.
- Durante a operação em produção: Garante estabilidade contínua e rápida detecção de falhas em produção.

Regra ISTQB: quanto mais cedo testamos, mais barato é corrigir.

Plano de Testes (Test Plan) — visão prática



Tipos de Teste — quando e por quê



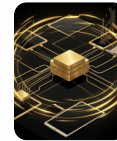
Teste de Unidade

Valida
funções/módulos
isolados; rápida
execução e baixo
custo de correção.



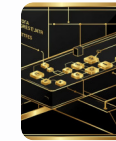
Teste de Integração / Interface

Verifica
comunicação entre
componentes e APIs.



Teste de Sistema / Não funcional

Valida
comportamento do
sistema completo:
desempenho,
segurança e
usabilidade.



Teste de Regressão / Automação

Garante que
mudanças não
quebrem
funcionalidades;
ideal para CI/CD.

Metodologia de Teste

É o conjunto organizado de processos, técnicas e atividades que guiam o planejamento, o design, a execução e a avaliação dos testes de software, visando garantir que o produto atenda aos requisitos especificados e seja entregue com qualidade.

– *ISTQB® Glossary*, versão mais recente

Metodologia de Teste	Descrição	Objetivo Principal
Teste Exploratório	Testes realizados sem roteiro rígido, com exploração guiada.	Encontrar falhas inesperadas e novas ideias.
Teste de Caixa Preta	Testa o software sem conhecer o código interno.	Validar saída a partir das entradas.
Teste de Caixa Branca	Testa o software com conhecimento do código-fonte.	Avaliar lógica interna e cobertura de código.
Teste de Caixa cinza	Testa o software combinando caixa preta e branca	Validar saída a partir das entradas e a lógica interna e cobertura de código.

Tipo de Teste

“Um grupo de atividades de teste direcionadas a testar um componente ou sistema com foco em um objetivo específico de teste, como verificar uma funcionalidade específica ou uma característica de qualidade.”

— Glossário ISTQB

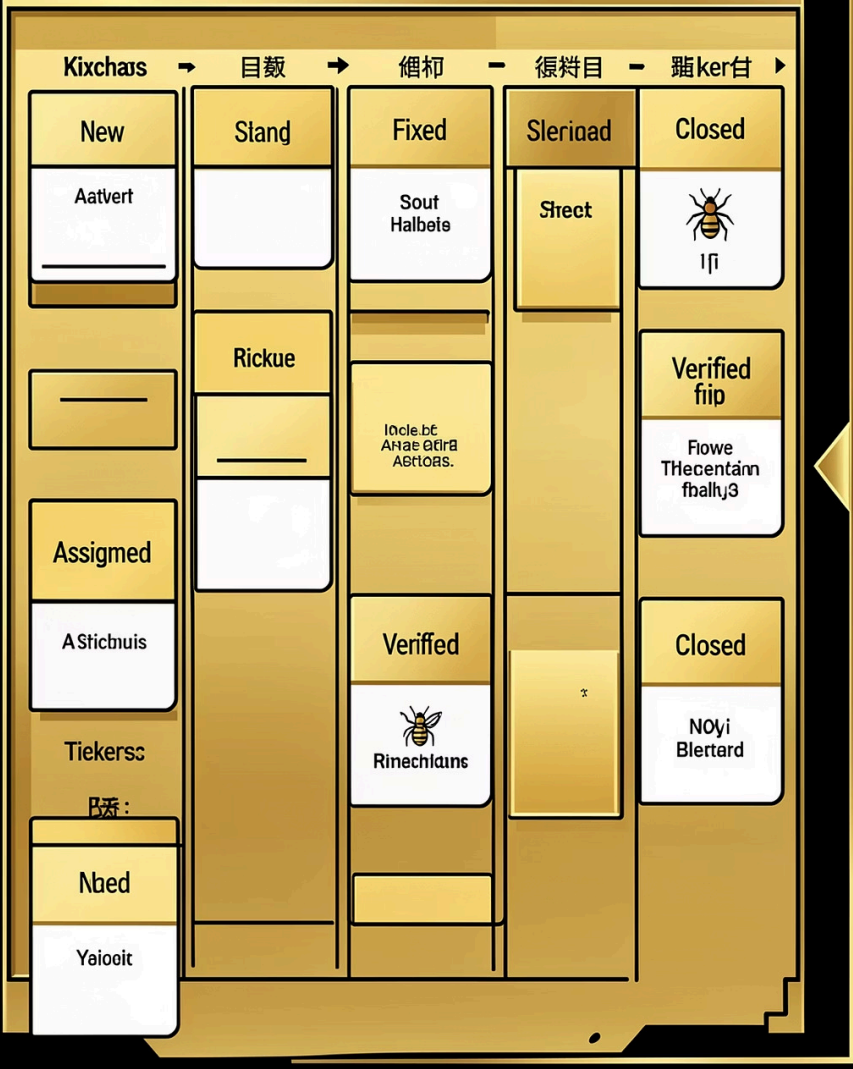
Tipo de Teste	Descrição	Objetivo
Teste de Unidade	Testa individualmente componentes do código (funções, classes, módulos).	Verificar se cada parte isolada funciona corretamente.
Teste de Integração	Testa a interação entre unidades ou módulos.	Garantir que componentes funcionem em conjunto.
Teste de Interface	Verifica a comunicação entre sistemas, APIs ou módulos.	Assegurar troca correta de dados entre partes.
Teste de Sistema	Avalia o sistema completo, integrado e funcional.	Validar se o sistema atende aos requisitos.
Teste Funcional	Verifica se as funcionalidades fazem o que foi especificado. E2E	Testar comportamentos esperados.
Teste Não Funcional	Avalia atributos como desempenho, usabilidade, escalabilidade, etc.	Medir qualidade além da funcionalidade.
Teste de Regressão	Reexecuta testes após mudanças no código.	Garantir que nada foi quebrado com alterações.
Teste de Aceitação	Realizado pelo cliente ou usuário para validação final.	Confirmar conformidade com os critérios do negócio.
Teste de Segurança	Verifica vulnerabilidades e proteção contra acessos indevidos.	Proteger dados e garantir integridade do sistema.
Teste de Usabilidade	Avalia a facilidade de uso e compreensão do sistema pelo usuário final.	Melhorar a experiência do usuário.
Teste de Instalação	Testa o processo de instalação em diferentes ambientes.	Garantir que o sistema possa ser instalado corretamente.
Teste de Componentes	Verificam se um componente visual isolado (por exemplo, um botão, formulário ou card) funciona corretamente — tanto em lógica quanto em interface.	Garantir que componentes funcionem.

Execução de Teste

O processo de executar um teste sobre o componente ou sistema sob teste, produzindo resultados reais.

– Glossário ISTQB

Forma de Execução	Descrição	Quando Utilizar
Teste Manual	O testador executa os testes passo a passo, com ou sem roteiro documentado.	Útil para testes exploratórios, usabilidade ou baixa repetição.
Teste Automatizado	Casos de teste são executados automaticamente por ferramentas/scripts.	Ideal para testes repetitivos, regressões, e CI/CD.
Teste Semi-Automatizado	Combina execução manual com apoio de ferramentas (ex: preenchimento automático, captura de evidências).	Quando a automação completa não é viável ou desejável.



Termos essenciais e fluxo de falhas

Erro → Defeito → Falha. Exemplo de fluxo: Erro humano gera Defeito no código; ao executar, ocorre Falha; registra-se no Test Log; reporta-se via Defect Report para correção.

- ❏ Use registros claros e passos de reprodução no relatório de falha para acelerar correção.

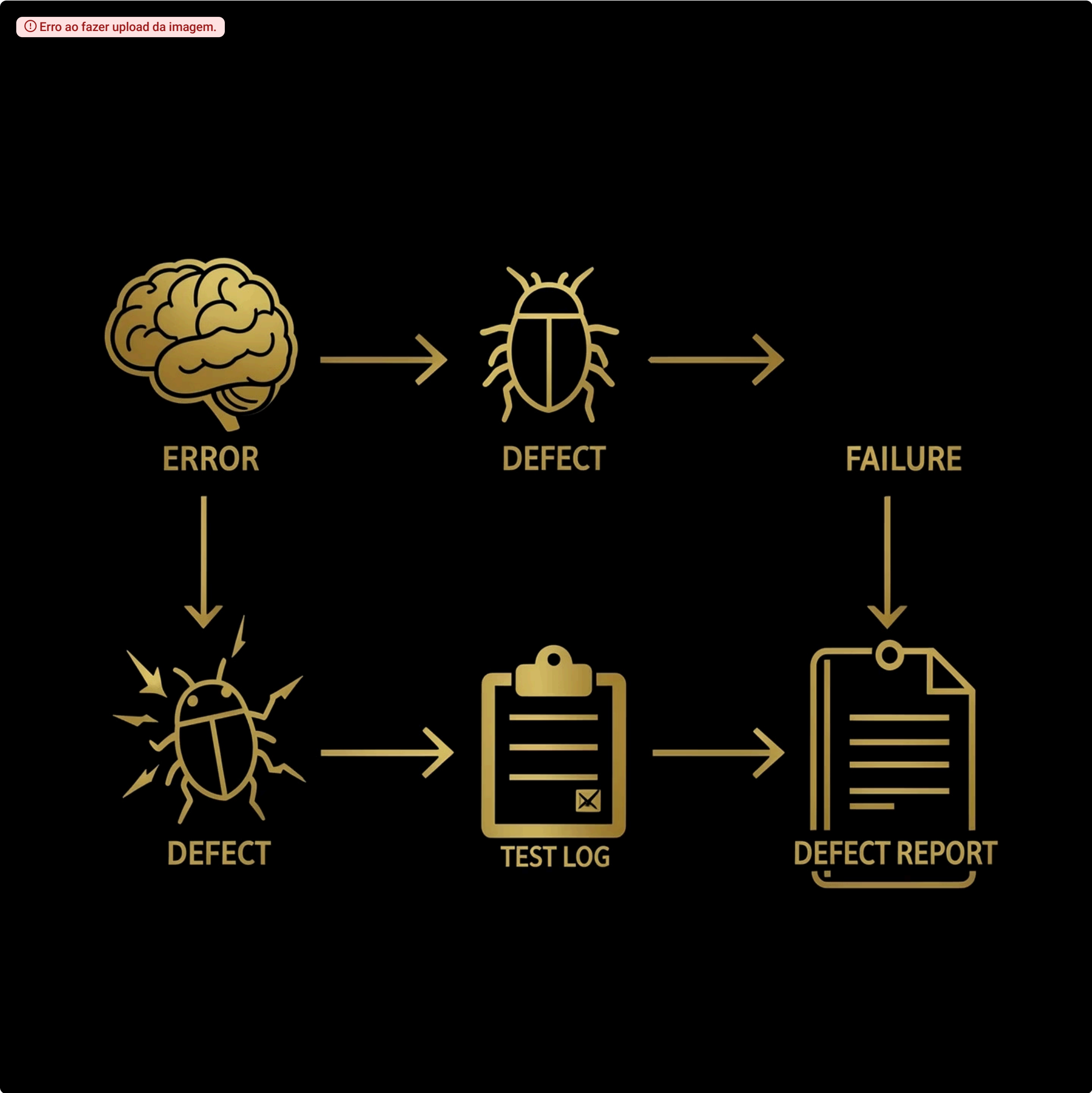
Falhas nos testes

🧠 Erro → 🐛 Defeito → 💣 Falha → 📋 Registro de Teste → 📝 Relatório de falha

Termo	Definição ISTQB	Comentário
Defeito (Defect / Bug / Fault)	Uma imperfeição no componente ou sistema que pode causar falha ao ser executado.	Normalmente introduzido por erro humano durante o desenvolvimento.
Falha (Failure)	A manifestação de um defeito durante a execução, resultando em comportamento incorreto do software.	Ex: sistema trava, resultado errado, resposta inesperada.
Erro (Error)	Uma ação humana que produz um resultado incorreto.	Causa raiz de um defeito. Ex: lógica errada no código.
Registro de Teste (Test Log)	Registro detalhado dos eventos ocorridos durante a execução dos testes.	Inclui passos executados, resultados obtidos, status de sucesso/falha.
Relatório de falha (Defect Report)	Documento que descreve um defeito encontrado, com informações para reprodução e correção.	Também chamado de “bug report” ou “ticket”.

Falhas nos testes

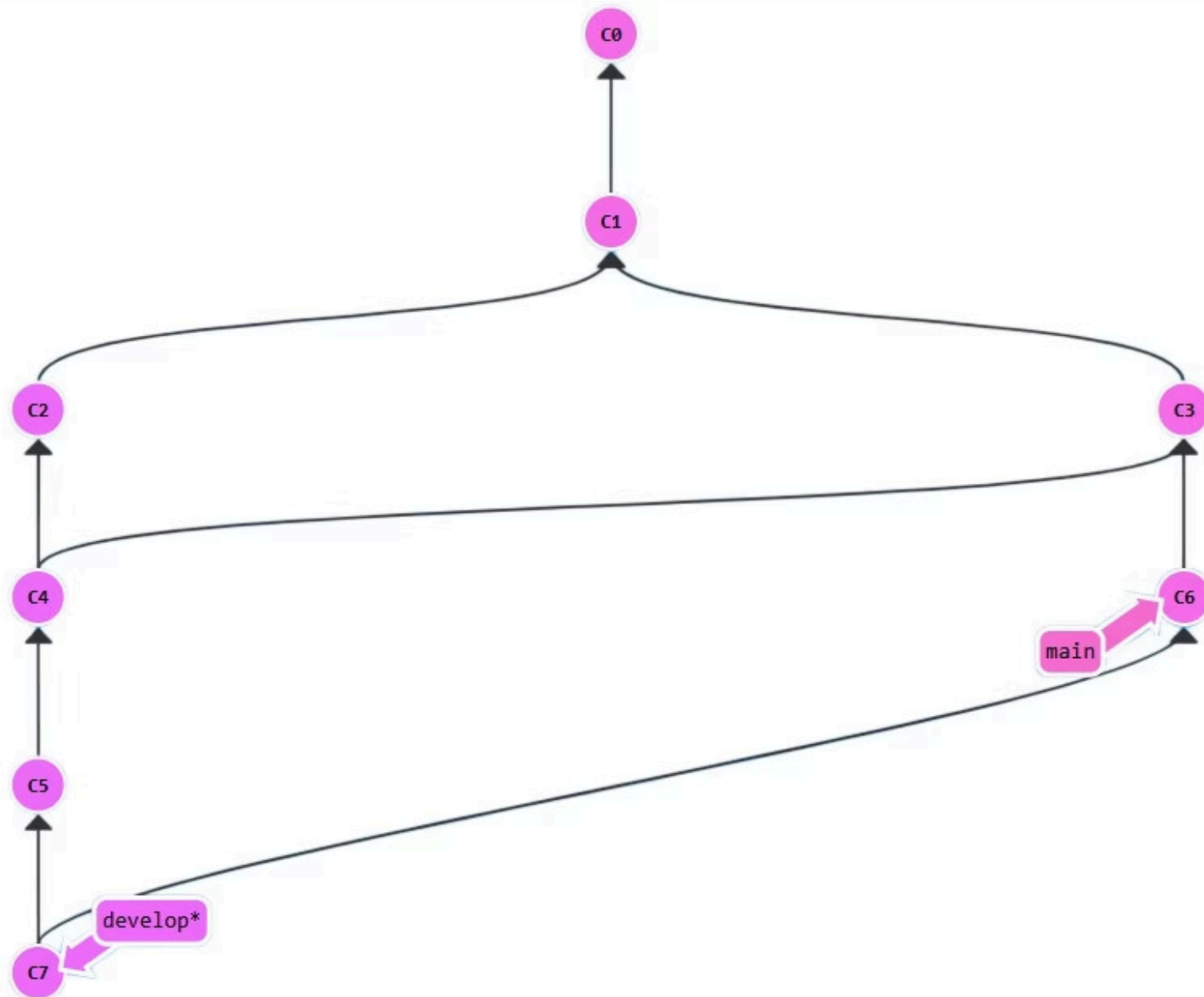
🧠 Erro → 🐛 Defeito → 💥 Falha → 📋 Registro de Teste → 📄 Relatório de falha



Controle de versão

Controle de versão (também conhecido como **versionamento de código**) é um **sistema que registra, gerencia e acompanha todas as alterações feitas em arquivos ao longo do tempo**, permitindo que múltiplas versões coexistam de forma organizada e segura.

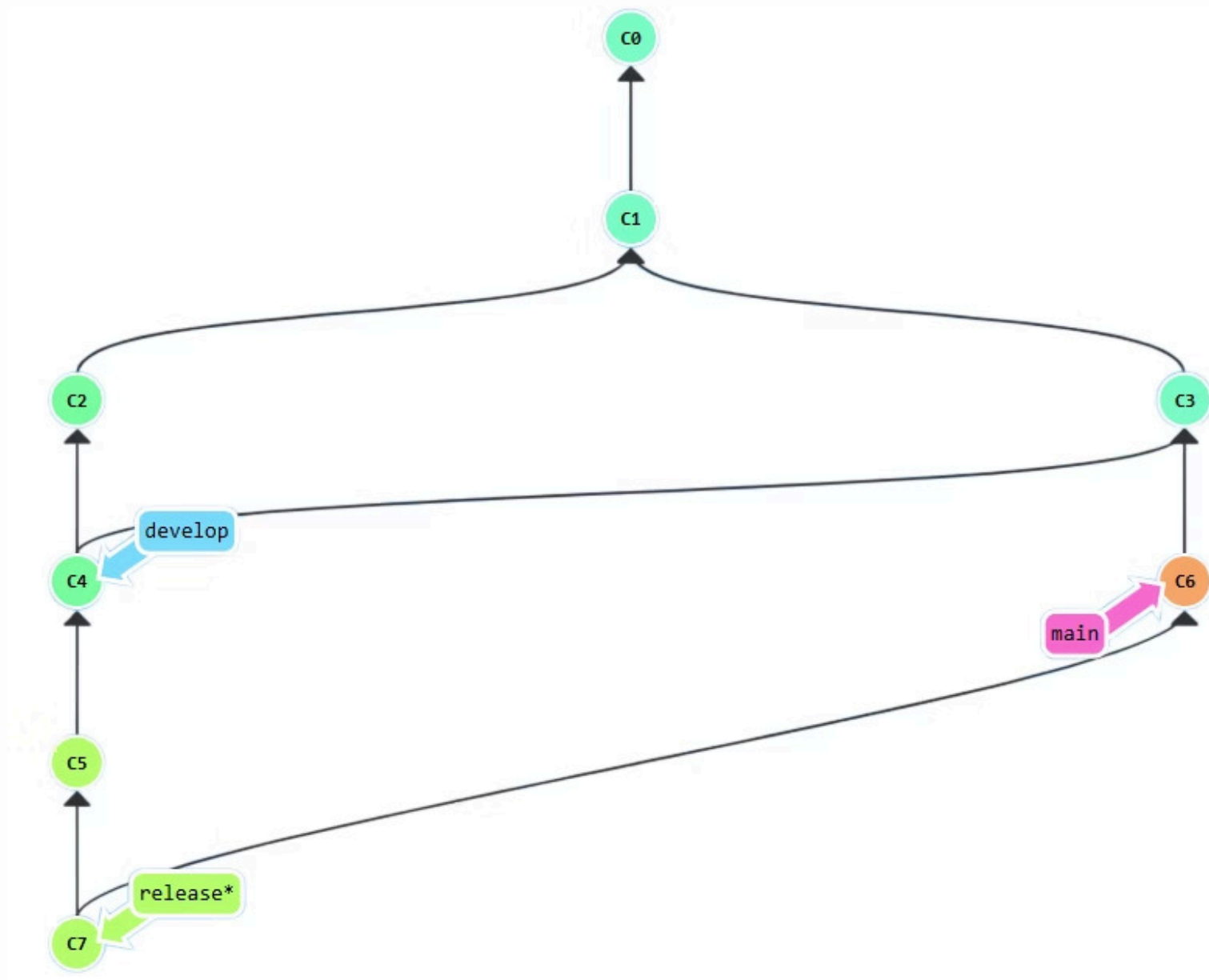
Learn Git Branching: <https://learngitbranching.js.org>



Ramificação do projeto

Ramificação do projeto, no contexto de controle de versão, é o processo de **criar linhas independentes de desenvolvimento** chamadas de **branches** (ramificações), que permitem trabalhar em novas funcionalidades, correções ou testes **sem afetar a base principal do código**.

Learn Git Branching: <https://learngitbranching.js.org>



Ciclo de Vida da Aplicação

“O Ciclo de Vida da Aplicação é o conjunto estruturado de fases pelas quais um software passa desde sua concepção até seu encerramento, incluindo planejamento, desenvolvimento, testes, implantação, manutenção e desativação.”

— *ISTQB® Glossary*, versão mais recente

Fase	Descrição
1. Levantamento de Requisitos	Identificação das necessidades do cliente e definição do escopo do sistema.
2. Análise	Estudo de viabilidade técnica e levantamento de requisitos detalhados.
3. Projeto (Design)	Arquitetura e modelagem técnica da aplicação (UI, banco, lógica, etc.).
4. Implementação	Codificação da aplicação conforme os requisitos e o design definidos.
5. Testes	Verificação e validação do software, com identificação e correção de defeitos.
6. Implantação (Deploy)	Entrega do software ao ambiente de produção e validação final.
7. Manutenção	Correções, atualizações e adaptações do sistema já em uso.
8. Encerramento	Descontinuação formal do sistema, migração de dados, arquivamento.

Política de dados

A **política de dados** é um **conjunto de diretrizes, procedimentos e ferramentas** adotados por uma organização para garantir que **dados perdidos, corrompidos ou inacessíveis possam ser restaurados de forma segura e eficiente**.

Backup é o processo de **copiar e armazenar dados em local seguro**, com o objetivo de recuperá-los em caso de falha, corrupção ou exclusão.

Restore é o processo de **reverter um backup para recuperar dados perdidos ou corrompidos**.

DESAFIO

Desenvolver uma calculadora básica com quatro operadores.

Regras de Negocio:

- Efetuar as 4 operações básicas (Soma, Multiplicação, Subtração e Divisão).
- Apresentar os resultados.

Com base na aplicação criada, desenvolver um plano de testes utilizando como base o Slide 9 da apresentação.

Encaminhar o Plano de teste para **wellington.speransa@prof.sc.senac.br**

Testes unitários

Testes unitários (ou **unit tests**) são um tipo de teste automatizado usado para verificar se **pequenas partes isoladas do seu código** — chamadas de *unidades* — estão funcionando corretamente.

O que é uma "unidade"?

Normalmente, uma **unidade** é uma **função, método** ou **componente individual**. A ideia é testá-la **separadamente**, sem depender de outras partes do sistema.

Método

```
function somar(a, b) {  
    return a + b;  
}
```

Teste do método

```
testar("Soma de positivos", () => {  
    if (somar(3, 4) !== 7) throw new Error("Esperado 7");  
});
```

Desafio

Utilizando a calculadora desenvolvida e o plano de testes elaborado.

Aplicar de forma prática o proposto pelo plano de teste.

Encaminhar o desafio através de link publico no GITHUB para **wellington.speransa@prof.sc.senac.br**

Testes de componente

Testes de componente verificam se um **componente visual isolado** (por exemplo, um botão, formulário ou card) **funciona corretamente** — tanto em **lógica** quanto em **interface**.

Eles vão além de testes unitários: testam **comportamento e renderização visual** de um pedaço da UI.

Estrutura de elementos

```
<form id="form">
  <input type="number" id="num1"/>
  <input type="number" id="num2"/>
  <button type="submit">Calcular</button>
</form>
```

Teste do componente

```
testar("Formulário deve estar presente na página", () => {
  const form = document.getElementById("form");
  if (!form) throw new Error("Form não encontrado");
});
```

Desafio

Utilizando a calculadora desenvolvida e o plano de testes elaborado.

Aplicar de forma prática os testes de componentes sobre a calculadora

Atualizar diretório do GITHUB e reencaminhar e-mail com link para **wellington.speransa@prof.sc.senac.br**

Relatório de falha

Modelo profissional e direto de relatório de falhas (bug report)

Campo	Descrição
ID da Falha	BUG-2025-0012
Título	[BUG] Campo de e-mail aceita formato inválido no cadastro
Data / Hora	15/05/2025 14:32
Reportado por	Alex Silva – Equipe Front-end
Ambiente	Windows 11 / Chrome 124 / Homologação
Descrição da Falha	Campo de e-mail aceita texto inválido (sem "@") e envia o formulário.
Comportamento Esperado	O campo deve validar e impedir envio com e-mail inválido.
Passos para Reproduzir	1. Acesse /cadastro2. Preencha os campos3. Use e-mail inválido4. Envie
Evidências	Screenshot, log do console, vídeo curto (se aplicável)
Severidade	Alta
Reprodutibilidade	Sempre ocorre
Encaminhado para	Time de Front-End

Desafio 4 — Evolução e Versionamento da Calculadora

Objetivo:

Realizar a evolução da calculadora desenvolvida nas atividades anteriores, adicionando a operação de porcentagem (%), aplicando conceitos de manutenção de software, testes e versionamento.

Atividade:

01

Implementar operação %

Adicionar a funcionalidade de porcentagem na calculadora.

02

Atualizar plano de testes

Incluir novos casos de teste para a operação de porcentagem.

03

Executar testes

Garantir que todas as operações (incluindo a nova) funcionem corretamente.

04

Atualizar projeto no GitHub

Subir o código e o plano de testes atualizados para o repositório.

Versionamento:

Publicar v1.2

Publicar a nova versão do sistema como **v1.2**.

Registrar Alterações

Registrar as alterações realizadas no projeto.

Entrega:

Enviar por e-mail o link público do repositório no GitHub contendo: wellington.speransa@prof.sc.senac.br

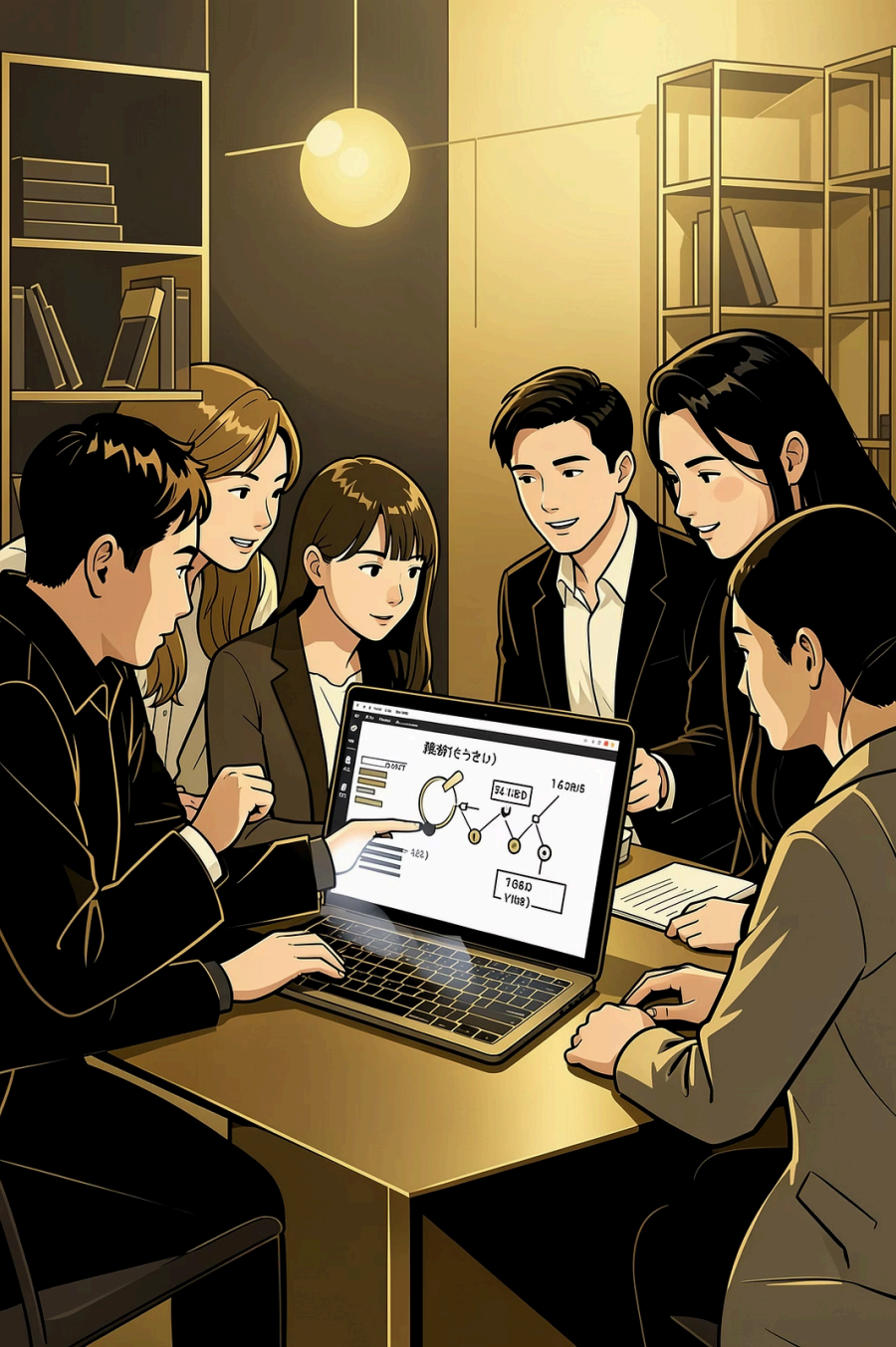
- Código atualizado
- Plano de testes atualizado
- Versão v1.2 publicada

Atividade complementar

Acessar ao site: <https://bugbank.netlify.app/>

Buscar por falhas, erros ou um defeito.

Elaborar documento realizando apontamentos sobre os erros encontrados.



Conclusões e próximos passos



Práticas

Integre testes desde requisitos; combine manual e automação.



Ferramentas

Familiarize-se com Selenium, Cypress, Jenkins e frameworks de testes unitários.



Carreira

Considere a certificação ISTQB para consolidar conhecimentos e destacar-se no mercado.

Atividade sugerida: analisar um caso bancário simples e criar um mini plano de testes (1 página) em dupla.